

COMP 8347: Internet Applications and Distributed Systems
Summer 2026 LAB #5

PART 1: Create views for your app in *views1.py*

1. Create *index* view. This will be the view for the main landing page of your app. It should show a list of the *books* that are in your db.

a. In your *myapp* dir create a new Python file *views1.py* and edit this file as follows:

```
# Import necessary classes
from django.http import HttpResponse
from .models import Publisher, Book, Member, Order

# Create your views here.
def index(request):
    response = HttpResponse()
    booklist = Book.objects.all().order_by('title')[:10]
    heading1 = '<p>' + 'List of available books: ' + '</p>'
    response.write(heading1)
    for book in booklist:
        para = '<p>' + str(book.id) + ': ' + str(book) + '</p>'
        response.write(para)
    return response
```

b. Create *urls1.py* under your *myapp* dir and edit it as follows:

```
from django.urls import path
from myapp import views1

app_name = 'myapp'

urlpatterns = [
    path('', views1.index, name='index'),
]
```

c. In your project's *urls.py* import **include** and **path** from **django.urls** then add the *path*:

```
path('myapp/', include('myapp.urls1')),
```

d. Start your server and go to **myapp**. You should see a list of *books*.

e. Update the *index* view function, so it displays a list of books ordered by the *primary key*. and then displays the list of Publishers as well. Each item in the list should show the *name* of the Publisher and its *city*. The list should be sorted alphabetically in descending order of their *city* name.

2. Create *about* view. Define another function `about(request)` in your *views1.py* file. When the user goes to url **myapp/about** it should display the following text: “**This is an eBook APP.**” You should also update *myapp/urls1.py* with the suitable pattern.

3. Create *detail* view. Define another function `detail(request, book_id)` in your *views1.py* file. When the user goes to url **myapp/book_id** it should display the *title* (in uppercase), *price* (with a '\$' in front of the price) and *publisher* for that book. You should also update *myapp/urls1.py* with the suitable pattern. Capture the `book_id` from the url and pass it as an argument to `detail(request, book_id)`.

COMP 8347: Internet Applications and Distributed Systems
Summer 2026 LAB #5

4. If a user types a url `myapp/book_id` and that `book_id` does not exist, the function `detail(request, book_id)` should return a **Page not found (404)** error.

- HINT: Import and use `get_object_or_404()` from `django.shortcuts`.

What difference you see in the `detail` page when you set `DEBUG = False` the file `settings.py`